Examination Result and Grace-Mark Validation

Problem statement

Process marks in multiple subjects, apply subject-wise pass criteria and a grace-mark policy, and generate final classifications without allowing any subject mark to exceed its maximum.

Assumptions and rules

• Each subject is represented by (name, marks_obtained, maximum_marks, pass_mark).

• Grace marks may be used only to raise a subject that is short of its pass mark, and the total grace used cannot exceed the policy limit.

• Grace is applied only when the subject is within the permitted shortfall. Final marks are capped at the subject maximum.

• A student passes only if every subject meets its pass mark after permitted grace.

Algorithm

1. Read the subject records and the maximum total grace marks.

2. Set grace_used to zero and total_obtained and total_maximum to zero.

3. For each subject, validate that marks and pass mark are within the maximum.

4. If marks are below pass mark, calculate the shortfall.

5. If the shortfall is within the allowed per-subject shortfall and enough total grace remains, add grace and update grace_used.

6. Cap the adjusted mark at the subject maximum; update totals and check the subject result.

7. After all subjects, calculate percentage and assign PASS / FAIL and a classification.

8. Display each subject's original mark, grace, final mark, and overall result.

Pseudocode

```
START\nINPUT subjects, total_grace_limit, per_subject_grace_limit\ngrace_used = 0; all_pass = TRUE\nFOR each subject\n  VAL
```

Flowchart

```
[Start]\n   |\n[Read subjects and grace limits]\n   |\n[Validate subject marks]\n   |\n[For each subject: calculate shortfa
```

Python solution

The program uses beginner-level Python constructs from the assigned syllabus: variables, data types, operators, conditions, loops, strings, lists, tuples, and unpacking where appropriate.

```
subjects = [
    ("Mathematics", 38, 100, 40),
    ("Physics", 67, 100, 35),
    ("Chemistry", 72, 100, 35),
    ("English", 81, 100, 40)
]

total_grace_limit = 5
per_subject_grace_limit = 3
grace_used = 0
total_obtained = 0
total_maximum = 0
all_pass = True

print("SUBJECT RESULT")
for name, marks, maximum, pass_mark in subjects:
    if marks < 0 or marks > maximum or pass_mark < 0 or pass_mark > maximum:
        print("Invalid data for", name)
        all_pass = False
        continue
```

```
    grace = 0
    shortfall = pass_mark - marks
    if shortfall > 0:
        if shortfall <= per_subject_grace_limit:
            if grace_used + shortfall <= total_grace_limit:
                grace = shortfall
                grace_used = grace_used + grace

    final_mark = marks + grace
    if final_mark > maximum:
        final_mark = maximum

    if final_mark < pass_mark:
        all_pass = False

    total_obtained = total_obtained + final_mark
    total_maximum = total_maximum + maximum
    print(name, "Original:", marks, "Grace:", grace,
          "Final:", final_mark)

if total_maximum > 0:
    percentage = total_obtained / total_maximum * 100
else:
    percentage = 0

if all_pass and percentage >= 75:
    classification = "DISTINCTION"
elif all_pass and percentage >= 60:
    classification = "FIRST CLASS"
elif all_pass and percentage >= 50:
    classification = "SECOND CLASS"
elif all_pass:
    classification = "PASS"
else:
    classification = "FAIL"

print("Grace used:", grace_used)
print("Total:", total_obtained, "/", total_maximum)
print("Percentage:", round(percentage, 2))
print("Final classification:", classification)
```

Sample output

```
Mathematics Original: 38 Grace: 2 Final: 40\nPhysics Original: 67 Grace: 0 Final: 67\nChemistry Original: 72 Grace: 0 Final
```

Explanation / observations

The example gives a maximum of 3 grace marks per subject and 5 overall. The grace policy is explicit and can be changed by editing the two limit variables.

Conclusion

This solution models the stated case using a clear sequence of input, processing, decisions, repetition, and output. The assumptions and sample data can be changed to test other cases.